Writing our own neural network: regression of the Runge function and classification of MNIST

Pietro Monacelli, Georgios Kotrotsios, Giorgio Spreafico, and Elia Valenti
FYS-STK3155/4155, University of Oslo, Norway
(Dated: November 2026)

Neural networks learn features from the raw input; whether this beats hand-made features depends on the problem. We wrote our own feed forward network, verified it against automatic differentiation and Scikit-Learn, and trained it with gradient descent, RMSprop and Adam. On 100 noisy samples of the Runge function, one hidden layer of 50 sigmoid nodes lowered the test mean squared error from 0.0187 ± 0.0048, for the best OLS polynomial of our previous project, to 0.0133 ± 0.0048 (paired difference -0.0054 ± 0.0028), and the error to the true function from 0.00572 to 0.00106; larger networks, other activations and penalties gained less than one standard error. On the full MNIST set, one hidden layer of 256 ReLU nodes reached a test accuracy of 0.9806 ± 0.0014, against 0.9241 ± 0.0026 for logistic regression. The network is the better tool, clearly on images and narrowly on the Runge function, whose peak the polynomials miss, but it needs far more parameters.

I. INTRODUCTION

In our first project we fitted the Runge function with polynomials, whose powers of x we had chosen ourselves [1]. The Runge function shows the limit of this choice: a low degree misses its narrow peak and a high degree oscillates near the ends of the interval [1], the bias and the variance of Ref. [2, Sec. 2.10]. A neural network instead learns its own features from the raw input [3]. One hidden layer can approximate any continuous function on a compact domain if its activation is not a polynomial [2, Sec. 8.4], [4], but "Representability is not learnability" [2, Sec. 8.4.4]. We therefore ask when a network is worth its cost compared with features chosen by hand.

We study two problems. The first is the Runge function, with the data and the ordinary least squares (OLS), Ridge [5] and Lasso [6] benchmarks of Ref. [1]; for a smooth function of one variable, OLS with a well chosen degree is cheap and hard to beat [3]. The second is the classification of the handwritten digits of MNIST [7], whose right features nobody knows; guessing gives about 10 %, logistic regression about 92 % and convolutional networks above 99 % [3]. Logistic regression is our network without hidden layers, so this comparison measures what a hidden layer adds.

We wrote a feed forward neural network (FFNN) in NumPy [8], with flexible layers, nodes and activations, the mean squared error or the cross entropy as cost, L_1 or L_2 penalties and backpropagation [9]. We verified it against automatic differentiation in JAX [10], the closed forms of Ref. [1] and Scikit-Learn [11], and trained it with the gradient methods of Ref. [1]: plain and stochastic gradient descent, RMSprop [12] and Adam [13]. We chose the hyperparameters on maps of two parameters at a time, by cross-validation or on validation data, and used each test set once. The network, its training and the scans are ours, adapted from the course code [14] and Project 1; Scikit-Learn provided the splits, the folds, the polynomial features and the reference models, and JAX only the checks. Our use of large language models is declared in Appendix C.

Section II presents the methods, Sec. III the code and its tests, Secs. IV and V the results on the Runge function and on MNIST, and Sec. VI their evaluation and our conclusions. The notebook that produces every number and figure of this report, executed from top to bottom, the Python script exported from it and the figures are in our repository [15]; we refer to its sections as (notebook, Sec. b.6), where the number is the step and the letter the part of the project text: a) costs and activations, b) regression network, c) architecture, d) penalties, e) classification, f) evaluation.

II. THEORY AND METHODS

A. Cost functions and matched pairs

Each cost is the negative log-likelihood of a model of the target [2, Secs. 3.6, 5.3, 5.6 and Eqs. (5.11)–(5.13)], so it follows from the type of target: Gaussian noise gives the mean squared error (MSE), suited to the Runge function; a Bernoulli target gives the binary cross entropy (BCE); a categorical target gives the multiclass cross entropy (CE), suited to MNIST. For n samples with targets y_i, or one hot rows y_{ik} for K classes, the costs are

$$C_{\mathrm{MSE}} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2 + P, \tag{1}$$

$$C_{\mathrm{BCE}} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \ln p_i + (1-y_i)\ln(1-p_i)\right] + P, \tag{2}$$

$$C_{\mathrm{CE}} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik} \ln a_{ik} + P, \tag{3}$$

where $\hat{y}_i$ is the network output, $p_i = \sigma(z_i)$ and the softmax $a_{ik} = e^{z_{ik}} / \sum_j e^{z_{ij}}$ are probabilities computed from the output pre-activations z, and $P = \lambda \sum_l \|\boldsymbol{W}^l\|_2^2$ (L_2)

or $\lambda \sum_l \|\boldsymbol{W}^l\|_1$ (L_1) penalises the weight matrices $\boldsymbol{W}^l$ of all layers. With $\sigma' = \sigma(1-\sigma)$ and the Jacobian of the softmax with $\sum_k y_{ik} = 1$ [2, Eqs. (5.5), (5.17), (5.29), (8.69)], the derivatives are

$$\frac{\partial C_{\mathrm{MSE}}}{\partial \hat{y}_i} = \frac{2}{n}(\hat{y}_i - y_i), \qquad \frac{\partial C_{\mathrm{BCE}}}{\partial z_i} = \frac{1}{n}(p_i - y_i),$$
$$\frac{\partial C_{\mathrm{CE}}}{\partial z_{ij}} = \frac{1}{n}(a_{ij} - y_{ij}), \tag{4}$$

and the penalty adds $2\lambda w$ or $\lambda\,\mathrm{sign}(w)$ to the derivative with respect to each weight w.

The data term is the mean squared residual, or minus the mean log-probability of the correct class k_i, since with one hot targets only $-\ln a_{i,k_i}$ survives. L_2 shrinks every weight in proportion to its size; L_1 pulls the weights towards zero with constant strength and, at the exact minimum of the Lasso, sets some of them to zero [2, Sec. 3.9]. The parameter λ balances fitting the data against small weights, and $1/n$ makes the cost an average. The BCE is derived for completeness; only the MSE and the CE are used [3].

Analytic and automatic gradients disagree when a convention differs [16], so we state ours. The MSE has no factor 1/2, unlike Ref. [2, Eq. (8.32)], so its derivative carries $2/n$. The penalty acts on the weights only, as the intercept is not penalised [2, Secs. 3.13, 8.12], although the summary of the project text [3] also mentions the biases. At $w = 0$ we use $\mathrm{sign}(0) = 0$, a valid element of the subgradient $[-1, 1]$ of $|w|$, as is the 1.0 that JAX returns [16], [2, Sec. 4.14.7]. Without hidden layers and with a linear output, our cost is that of Ref. [1] with the same λ, which Sec. III uses for verification.

For the three matched pairs, the identity output with the MSE, the sigmoid output with the BCE and the softmax output with the CE, the output error is the prediction minus the target [2, Eq. (8.69), Sec. 5.3],

$$\boldsymbol{\delta}^L \equiv \frac{\partial C}{\partial \boldsymbol{z}^L} = c\,(\boldsymbol{a}^L - \boldsymbol{y}), \tag{5}$$

with $c = 2/n$ for the MSE and $c = 1/n$ for the two cross entropies. A mismatched pair, the sigmoid output with the MSE, carries instead the factor $\sigma'(z^L) \le 1/4$, which vanishes exactly where the network is confidently wrong (Fig. 1): at $z = -5$ with target 1, $|\delta^L|$ is 0.9933 with the cross entropy and 0.0132 with the MSE, a ratio of 75 (notebook, Sec. a.5); the course notebook finds 150 with the half squared error [14]. The network would learn almost nothing from its worst mistakes, so we classify with the cross entropy. The matched pairs also spare the code the Jacobian of the softmax, and the factor $\sigma' \le 1/4$ returns inside a deep network as the vanishing gradient of Sec. IV C.

B. Activation functions

Table I lists the hidden activations; the output layer uses the identity for regression and the softmax for classification. Without a non-linearity a network of any depth is a linear map [2, Sec. 8.2]. The sigmoid is smooth and bounded, which suits shallow networks, but every layer multiplies the gradient by a factor $\sigma' \le 1/4$ [2, Eq. (8.71)], so in deep networks the gradient of the first layers vanishes [2, Sec. 8.9]; its output is also not centred [2, Sec. 8.6.1]. The ReLU does not saturate for $z > 0$ [2, Sec. 8.6.2] and is the recommended default [2, Sec. 8.6.4]; it is piecewise linear, and one hidden layer of N nodes gives at most $N + 1$ linear pieces [2, Lemma 8.1]. Units whose input stays negative stop learning [2, Sec. 8.6.2]; we take $f'(0) = 0$. The leaky ReLU keeps the slope $\alpha = 0.01$ for $z < 0$, so its gradient is never exactly zero.

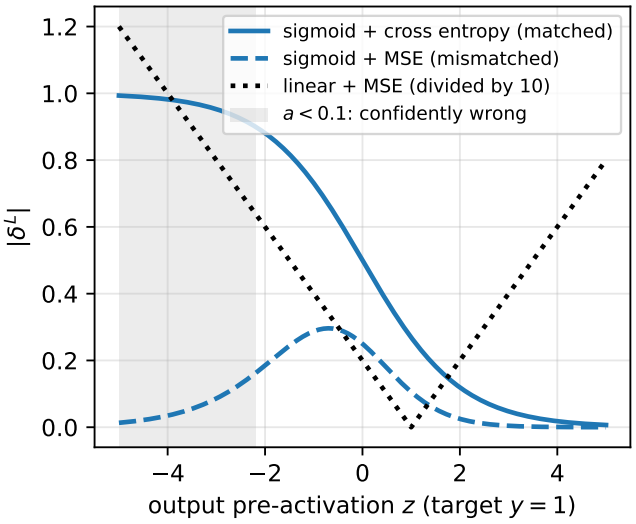


FIG. 1. The matched pair keeps the output error large where the network is confidently wrong, and the mismatched pair does not: at $z = -5$ with target 1, $|\delta^L| = 0.9933$ for the sigmoid with the cross entropy and 0.0132 for the sigmoid with the MSE (notebook, Sec. a.5, right panel of its figure).

C. Network, backpropagation and initialisation

We denote a network by the widths of its hidden layers: [50, 50] has two hidden layers of 50 nodes, and [784, 128, 10] also gives the input and the output sizes. With the samples in the rows of $\boldsymbol{A}^0 = \boldsymbol{X}$, the forward pass [2,

TABLE I. The derivative of the sigmoid is at most 1/4, while that of the ReLU and of the leaky ReLU is 1 for $z > 0$: the hidden activations [2, Eqs. (8.22)–(8.24)], with their derivatives and ranges.

	$f(z)$	$f'(z)$	range
sigmoid	$\sigma(z) = 1/(1+e^{-z})$	$\sigma(1-\sigma) \le 1/4$	$(0, 1)$
ReLU	$\max(0, z)$	1 if $z > 0$, else 0	$[0, \infty)$
leaky ReLU	z if $z > 0$, else αz	1, else $\alpha = 0.01$	$\mathbb{R}$

Eq. (8.21)], as in the course template [17], is

$$\boldsymbol{Z}^l = \boldsymbol{A}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l, \quad \boldsymbol{A}^l = f_l(\boldsymbol{Z}^l), \quad l = 1, \ldots, L, \quad (6)$$

with $\boldsymbol{W}^l$, $\boldsymbol{b}^l$ and f_l the weights, the biases and the activation of layer l. Backpropagation [2, Eqs. (8.65)–(8.68)] starts from $\boldsymbol{\delta}^L$ of Eq. (5) and runs backwards,

$$\boldsymbol{\delta}^l = \left(\boldsymbol{\delta}^{l+1}(\boldsymbol{W}^{l+1})^\mathsf{T}\right) \circ f_l'(\boldsymbol{Z}^l),$$
$$\frac{\partial C}{\partial \boldsymbol{W}^l} = (\boldsymbol{A}^{l-1})^\mathsf{T}\boldsymbol{\delta}^l + \frac{\partial P}{\partial \boldsymbol{W}^l}, \qquad \frac{\partial C}{\partial \boldsymbol{b}^l} = \sum_i \boldsymbol{\delta}_i^l, \quad (7)$$

with $\circ$ the element-wise product and $\boldsymbol{\delta}_i^l$ the row of sample i. It is the chain rule in one backward sweep, at a cost of about two forward passes whatever the number of parameters [3]; it computes the gradient, and the optimiser uses it.

The design matrix is the standardised x alone: the hidden layers build the features. The output is the identity, since the centred target is real with both signs and the identity with the MSE is a matched pair. The weights are normal with variance $1/N_{l-1}$ for the sigmoid and $2/N_{l-1}$ for the ReLU family [2, Eq. (8.74)], [18], N_{l-1} being the number of nodes of the previous layer, so that the signal keeps its scale in the forward pass [2, Eq. (8.72)] and the sigmoid does not saturate; Glorot and Bengio [19] compromise between this condition and that of the backward pass [2, Eq. (8.73)]. The weights are random, since equal weights would keep the nodes equal. The biases start at 0.01, the value Ref. [2, Sec. 8.10] suggests for the ReLU; the weights already break the symmetry. We standardise x and centre y with the statistics of the training data alone, in every fold [2, Sec. 3.13], so that the mean of y is an unpenalised intercept.

D. Optimisers

Plain gradient descent (GD) steps by $-\gamma$, the learning rate, times the gradient on all the training data, the full batch [2, Eq. (4.15)]; stochastic gradient descent (SGD) uses minibatches of M samples, reshuffled at every epoch, one pass over the data [2, Eq. (4.37)]. RMSprop divides each step by the root of a running average of the squared gradients, with decay $\rho = 0.99$ [2, Eqs. (4.63)–(4.66)], [12]; Adam adds a running average of the gradients, the momentum, and corrects both averages for their start at zero, with decays $\beta_1 = 0.9$ and $\beta_2 = 0.999$ [2, Eqs. (4.67)–(4.72), Sec. 4.11], [13]; both add $\epsilon = 10^{-8}$ to the root. The cost of a network is not convex, so γ, the initialisation and M decide whether and where training converges [3], [2, Sec. 8.7]; we chose γ on validation data.

E. Data and protocols

Regression. The data are those of Ref. [1]: 100 points with x uniform in $[-1, 1]$ and $y = f(x) + \varepsilon$, where $f(x) = 1/(1 + 25x^2)$ is the Runge function and $\varepsilon \sim N(0, 0.1^2)$, seed 2026, split into 80 training and 20 test points. We selected the models by 5-fold cross-validation (CV) on the training points, with the same folds for polynomials and networks, and used the test points once. We report the CV and test MSE, each with its standard error (s.e.), the standard deviation of the five fold MSEs or of the 20 squared test errors over $\sqrt{5}$ or $\sqrt{20}$, and the noise-free MSE to f on 500 points of the training interval. Paired differences between two models are taken fold by fold or point by point, with their s.e. computed in the same way. The true f scores 0.0150 on the training and 0.0096 on the test points (notebook, Sec. b.5): these are the floors of the first two errors, not $\sigma^2 = 0.01$.

MNIST. MNIST [7] has 60000 training and 10000 test images of 28×28 pixels, flattened into 784 features divided by 255, with one hot targets; a stratified validation set of 10000 images is split from the training images. The scans trained on 10000 of the other 50000 images for 10 epochs; the final models trained on all 50000 for at most 50 epochs and kept the epoch with the highest validation accuracy (early stopping); the test set was used once. Minibatches had $M = 128$ [2, Table 4.3]. The accuracy [2, Eq. (5.68)] on n images has the standard error $\sqrt{p(1-p)/n}$ [2, Secs. 2.5, 10.12.6]. The confusion matrix counts how often each true digit is predicted as each digit; it gives the precision, the recall and their harmonic mean F_1 of every digit, averaged in the macro F_1 [2, Eqs. (5.67), (5.69), (5.70)]. The floor, "the score obtained for free" [2, Sec. 5.12], is the accuracy of always predicting the most frequent digit.

Algorithm 1 The training loop, `train_network` (notebook, Sec. b.3).

Require: parameters $\boldsymbol{\theta}$, data $(\boldsymbol{X}, \boldsymbol{Y})$ of n rows, update rule, γ, epochs E, batch size M ($M = n$: full batch)

```
t ← 0
for e = 1, ..., E do
    shuffle the rows; split them into batches of M
    for each batch do
        t ← t + 1; gradient g by Eqs. (5), (7)
        θ ← step of plain GD, RMSprop or Adam
    if θ is not finite then mark as diverged; stop
    record the monitored errors
    with early stopping, keep θ at the best validation error
with early stopping, restore the kept θ
```

III. IMPLEMENTATION AND VERIFICATION

The core of our code, written in NumPy [8] from the course code [14], [20] and Project 1 [1], is a class and a training function. The class `FFNN` keeps all weights and biases in one vector $\boldsymbol{\theta}$, with the $\boldsymbol{W}^l$ and $\boldsymbol{b}^l$ of every layer as views into it, so that the update rules of Project 1 act on $\boldsymbol{\theta}$ directly; it computes the forward pass, the backpropagation and the penalties (notebook, Sec. b.2). The

training function `train_network` is given in Algorithm 1 (notebook, Sec. b.3). Matplotlib [21] and seaborn [22] draw the figures. The folder `Code` of our repository [15] holds Project2.py Project2.ipynb.

Table II lists the tests. The agreement with `jax.grad` is at the level of rounding, and central finite differences, the weakest test, agree to a relative $3.5\cdot10^{-6}$ or better. A network without hidden layers, trained by plain GD with the learning rate γ^* set by the largest and the smallest eigenvalues of the Hessian [2, Eq. (4.26)], reproduces the closed form OLS and Ridge solutions of Ref. [1], which checks the whole chain of cost, backpropagation and update rule. Scikit-Learn's `MLPRegressor`, with the same hidden layer of 50 sigmoid nodes, lies within one standard error of our network (Table III). For numerical safety the sigmoid is computed as $\frac{1}{2}[1+\tanh(z/2)]$, the softmax subtracts its largest input before the exponential [2, Eq. (5.30)], and probabilities are clipped at 10^{-12}. The seed is 2026 everywhere except for the initial weights of the networks of the backpropagation checks (seed 1) and in the starter cell of the project text (`random_state=42`).

IV. RESULTS: REGRESSION OF THE RUNGE FUNCTION

A. Update rules and the comparison with OLS

Benchmark. Five-fold CV over the polynomial degrees 1 to 15 selects OLS of degree 8 (notebook, Sec. b.5) (CV MSE 0.0266 ± 0.0071, test MSE 0.0187 ± 0.0048, MSE to f 0.00572), the OLS model with the lowest test MSE in Ref. [1] and the benchmark the project text asks for [3]; no other degree lies within one standard error of it.

TABLE II. Our derivatives and backpropagation agree with `jax.grad` to within $8.9\cdot10^{-16}$ and with central finite differences to within a relative $3.5\cdot10^{-6}$, and the network without hidden layers, on polynomial features of degree 5, reproduces the closed form OLS and Ridge weights to within $2.8\cdot10^{-15}$ in norm (notebook, Secs. a.4 and b.4). Last row: Scikit-Learn's `MLPRegressor` with the settings of our [50], Adam, $\gamma=10^{-2}$, full batch, 1000 epochs, no penalty (Sec. b.7).

test	result
cost and activation derivatives, penalised gradients of the linear models, against `jax.grad`	$\le 8.9\cdot10^{-16}$
matched pairs (5) against `jax.grad`	$\le 1.4\cdot10^{-17}$
backpropagation against `jax.grad`, 36 combinations of output, activation and penalty	$\le 6.9\cdot10^{-17}$
backpropagation against central finite differences, 20 random draws of a parameter, step 10^{-6}	$3.8\cdot10^{-9}$ to $3.5\cdot10^{-6}$
no hidden layer against closed form OLS	$2.8\cdot10^{-15}$
no hidden layer against closed form Ridge, $\lambda=10^{-2}$	$8.0\cdot10^{-16}$
`MLPRegressor` [50], sigmoid: CV MSE, test MSE, MSE to f	0.0181 ± 0.0039, 0.0128 ± 0.0046, 0.00081

Update rules and learning rates. We trained [50] sigmoid networks for 1000 epochs with the three update rules, with the full batch and with minibatches of $M=8$, at learning rates from 10^{-4} to 1 (Fig. 2). Adam with the full batch reaches the lowest CV MSE, 0.0175 ± 0.0038 at $\gamma=10^{-2}$, and five of the six settings come within one standard error of it. The exception, plain GD with the full batch, does best at $\gamma=0.1$ (0.0301 ± 0.0041, above the 0.0266 of OLS), stays between 0.0834 and 0.1031 for $\gamma\le3.2\cdot10^{-2}$, near the variance of y (0.0938), and diverges at $\gamma=1$. With the full batch, Adam stays between 0.0175 and 0.0184 from $\gamma=3.2\cdot10^{-3}$ to 0.32, while RMSprop gives 0.0313, 0.1253 and 0.0213 at $\gamma=10^{-2}$, $3.2\cdot10^{-2}$ and 0.1, as expected from the update rules: the step of plain GD is proportional to the gradient, that of Adam is at most about γ [2, Eq. (4.72)], and RMSprop at constant γ does not settle [2, Sec. 4.10]. At the best γ of each setting, the training MSE on all 80 points runs from 0.0160 (RMSprop, $M=8$) to 0.0636 (plain GD, full batch) after 100 epochs and from 0.0137 to 0.0176 after 5000 (notebook, Sec. b.6): minibatches make more updates per epoch [2, Eq. (4.38)] and converge in fewer epochs, but a 5-fold CV took 1.52 to 2.10 s with $M=8$ against 0.57 to 0.72 s with the full batch, on our machine. With a suitable γ every update rule works, Adam over the widest range; the rest of the regression study therefore uses Adam with the full batch, $\gamma=10^{-2}$ and a fixed budget of 1000 epochs, the set-up called TRAIN in Figs. 2 and 6, unless stated otherwise.

Networks against OLS. The four networks of Fig. 3, with γ retuned by CV to $3.3\cdot10^{-3}$ for [50, 50] and $3\cdot10^{-2}$ for [100, 100], score below OLS in CV, on the test set and with respect to f (Table III). Paired with OLS, the difference ranges from -0.0092 ± 0.0068 to -0.0098 ± 0.0071 in CV, with the network better in four or five of the five folds, and from -0.0051 ± 0.0027 to -0.0071 ± 0.0033 on the test points (notebook, Sec. b.7); among themselves, the four networks differ by less than one standard error. The polynomial does not reach the top of the peak (about 0.84, read from Fig. 3) and oscillates near $x=\pm1$, while the networks follow the peak: their advantage comes from their basis, with moderate evidence on 80 training and 20 test points.

B. Activation, depth and width

With one layer of 50 nodes, the best of five learning rates gives a CV MSE of 0.0175 for all three activations, at $\gamma=10^{-2}$ for the sigmoid and the ReLU and $3\cdot10^{-2}$ for the leaky ReLU, with a flat optimum between 0.0175 and 0.0183 from 10^{-2} to 0.1 (notebook, Sec. c.1). Each activation has its own γ, as in the course code [14], since

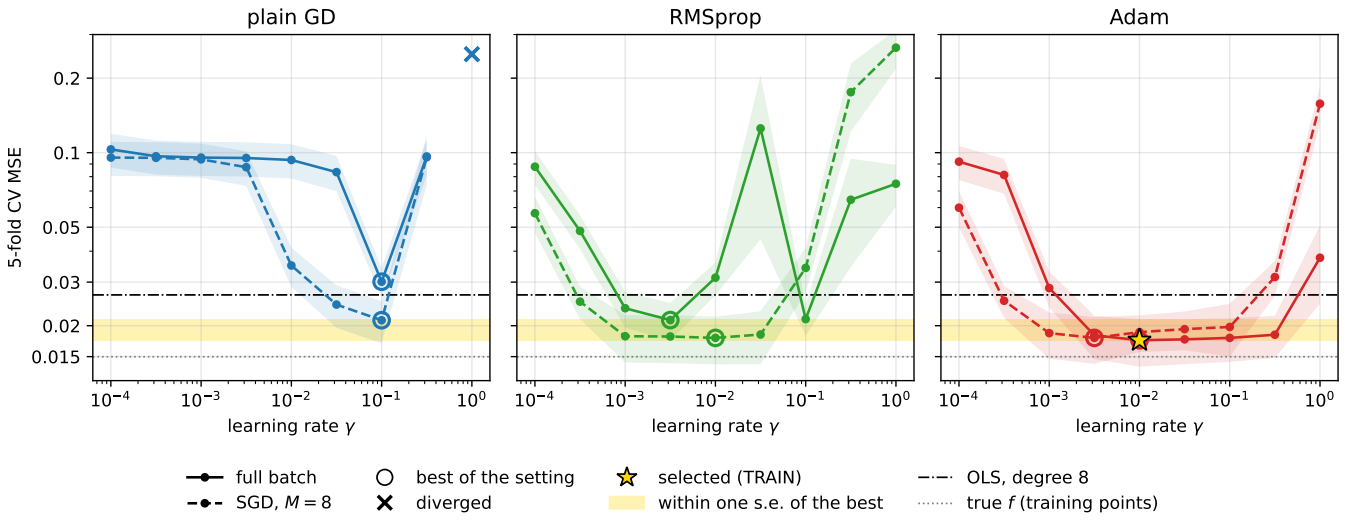


FIG. 2. For [50] sigmoid after 1000 epochs, five of the six settings of update rule and batch come within one standard error of the lowest CV MSE, 0.0175 ± 0.0038 for Adam with the full batch at $\gamma = 10^{-2}$ (star), and Adam does so over the widest range of learning rates, from $\gamma = 3.2 \cdot 10^{-3}$ to 0.32 with the full batch. Bands around the curves: one standard error over the five folds; seed 2026.

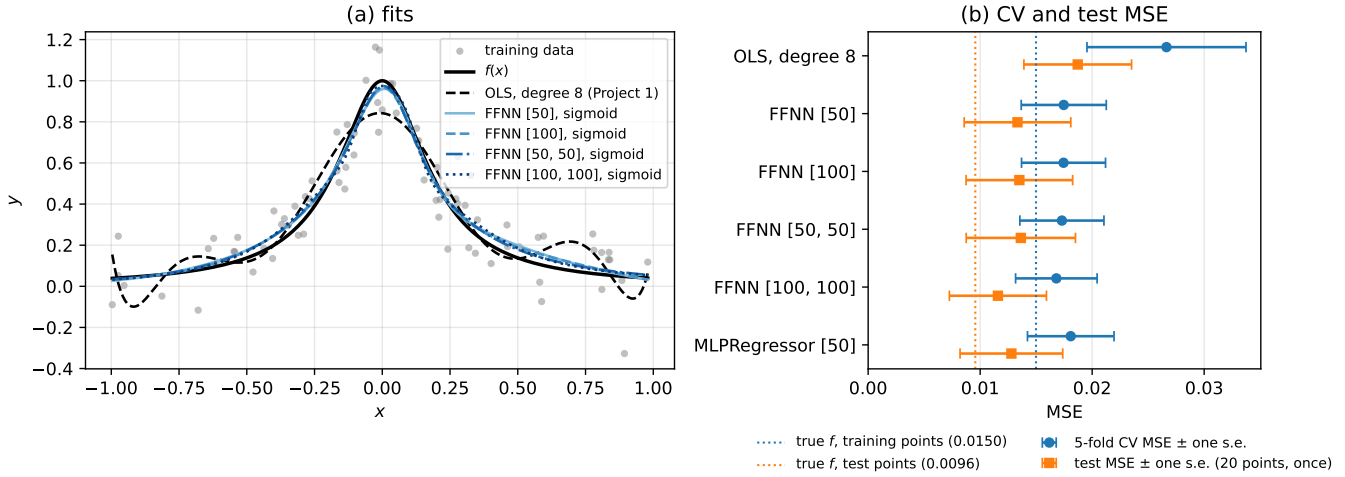


FIG. 3. Our four sigmoid networks follow the narrow peak that OLS of degree 8 misses (a) and score below OLS, with a CV MSE of 0.0168 to 0.0175 against 0.0266 and a test MSE of 0.0116 to 0.0136 against 0.0187, although their error bars overlap with those of OLS (b). Error bars: one standard error; seed 2026.

the size of the gradients depends on the activation and on the initialisation.

At these learning rates (Fig. 4) the best networks, [5, 5] sigmoid, [25] ReLU and [100, 100] leaky ReLU, agree within their standard errors (0.0171 to 0.0173). All 20 sigmoid networks lie within one standard error of their best, against 15 of 20 ReLU and 13 of 20 leaky ReLU networks. Narrow ReLU networks underfit (one hidden layer of 5 nodes: CV MSE 0.0293, training MSE 0.0251), and deep wide ones overfit (Sec. IV C). All three activations can represent f [2, Sec. 8.4], but the ReLU needs more nodes for its linear pieces (Sec. II B), and, as in Ref. [2, Table 8.2], the ReLU and the leaky ReLU cannot be told apart: the activation decides how robust the result is to the architecture, not the best network. The overfitting holds at these learning rates and 1000 epochs, since four layers of 50 ReLU nodes give 0.0188 at $\gamma = 10^{-3}$ (Sec. IV D), and the irregular cell [50, 50] ReLU (0.0237) is compatible with a single seed [2, Secs. 8.7, 8.15].

C. Overfitting and vanishing gradients

Signs of overfitting. Deep and wide ReLU networks show the signs of overfitting that the project text asks

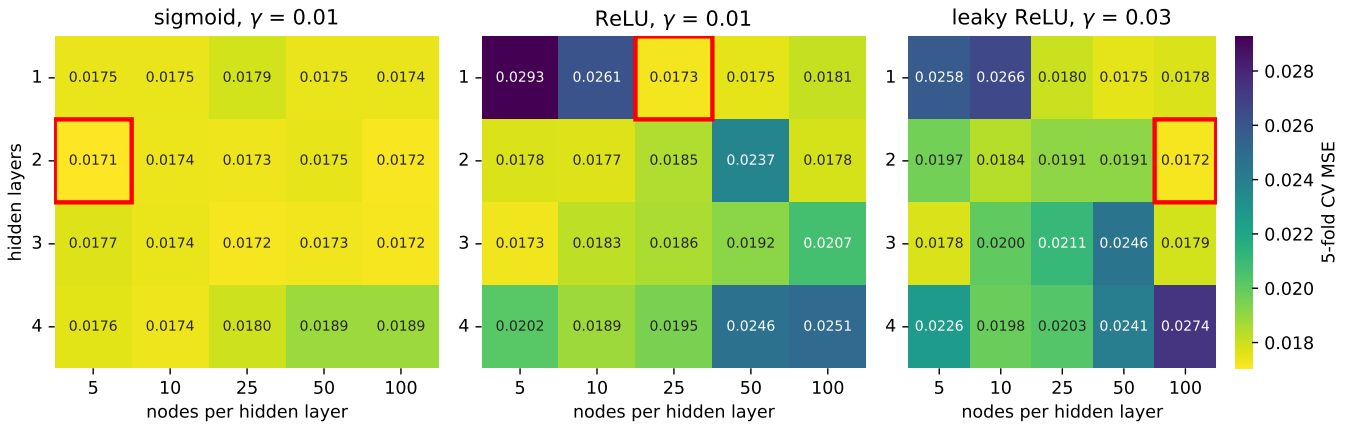


FIG. 4. The best cells of the three activations agree within their standard errors of 0.0034 to 0.0038, with 0.0171 for the sigmoid, 0.0173 for the ReLU and 0.0172 for the leaky ReLU, but all 20 sigmoid architectures lie within one standard error of the best, against 15 of 20 for the ReLU and 13 of 20 for the leaky ReLU. Red frames: the best cell of each activation; seed 2026.

about [3] (Fig. 5). The gap between CV and training MSE is small for the sigmoid (0.0175 and 0.0136 for [50], 0.0189 and 0.0116 for four layers of 100 nodes) and large for four layers of 100 ReLU nodes (0.0251 and 0.0123; 0.0274 and 0.0117 with the leaky ReLU). The validation MSE of that ReLU network is lowest at epoch 125 (0.0183) and rises to 0.0270 at 5000 epochs, while its training MSE falls to 0.0073; that of [50] sigmoid is lowest at epoch 1125 (0.0175). A validation error that rises while the training error still falls is overfitting [2, Secs. 4.7.8, 8.12], while a training MSE below the floor of 0.0150 alone is not, since even a correct linear model fits part of the noise [2, Eq. (3.35)]. The number of epochs thus plays the role of the polynomial degree of Project 1, and early stopping, a regularisation that costs nothing [3], [2, Sec. 8.12], would have kept epoch 125 for this network; we use it on MNIST, while the regression study keeps 1000 epochs.

Vanishing gradients. With six hidden layers of 50 nodes (Fig. 6), the norm of the weight gradient at initialisation is smaller in the first layer than at the output by a factor $6.5 \cdot 10^5$ for the sigmoid ($9.7 \cdot 10^{-6}$ against 6.4) and 12 for the ReLU family (notebook, Sec. c.4). Plain GD with $\gamma = 0.03$, the same for the three activations, takes the sigmoid network from a training MSE of 0.8735 to 0.0939 in 10 epochs and leaves it there, at about the variance of y (0.0938): it predicts the mean. The ReLU network reaches 0.0524, 0.0248 and 0.0138 after 10, 100 and 1000 epochs. Adam brings the sigmoid network to 0.0938 at epoch 100, keeps it there until about epoch 200 (read from the figure) and reaches 0.0117 after 1000 epochs. Every layer adds a factor f' that the initialisation does not remove (Sec. II B); plain GD steps in proportion to the gradient, so the first layers barely move [2, Sec. 8.9], while the step of Adam does not shrink with it [2, Sec. 4.11, Eq. (4.72)]. In our test, depth made the sigmoid network slow, and stuck at the mean with plain GD at $\gamma = 0.03$, but not untrainable, while the ReLU family trained with both update rules [Fig. 6(b), (c)]: for a deep network we would start from the ReLU.

D. L_2 and L_1 penalties

Penalty, depth and learning rate. We used the ReLU, within one standard error of the best activation (Sec. IV B), with 50 nodes per layer, where its deep networks overfit. At $\gamma = 10^{-2}$ [Fig. 7(a), (b)], 2 to 4 hidden layers without penalty have a lower training MSE than one layer (0.0109 to 0.0122 against 0.0136) but a higher CV MSE (0.0192 to 0.0246 against 0.0175). With $\lambda = 10^{-3}$ and either penalty their CV MSE falls to between 0.0172 and 0.0182, while with one layer no penalty helps; at $\lambda = 0.1$ the CV MSE of all depths lies between 0.0908 and 0.0954, near the variance of y: the network predicts the mean [2, Sec. 2.12]. A penalty removes variance [2, Sec. 2.10], so it helps only where the network overfits, and it makes the CV MSE almost independent of the depth: with a penalty, a network chosen too deep costs little. For four layers, where L_2 helps most [Fig. 7(c), (d)], the column $\lambda = 10^{-3}$ is the best at four of the five learning rates with both penalties, while without penalty the CV MSE rises from 0.0188 to 0.0360 as γ grows from 10^{-3} to 0.1: the penalty makes training less sensitive to γ. We selected $\gamma = 10^{-3}$ and $\lambda = 10^{-3}$ for both penalties (CV MSE 0.0173 ± 0.0038 with L_2, 0.0175 ± 0.0040 with L_1). This γ is the edge of the grid, where the CV MSE without penalty still falls (0.0246, 0.0212 and 0.0188 from $\gamma = 10^{-2}$ to 10^{-3}), and a smaller γ may itself act like early stopping, since each step of Adam is at most about γ.

Against Ridge and Lasso. We compared the penalised networks with the best Ridge and Lasso models of Ref. [1], refitted on our 80 training points (Fig. 8, Ta-

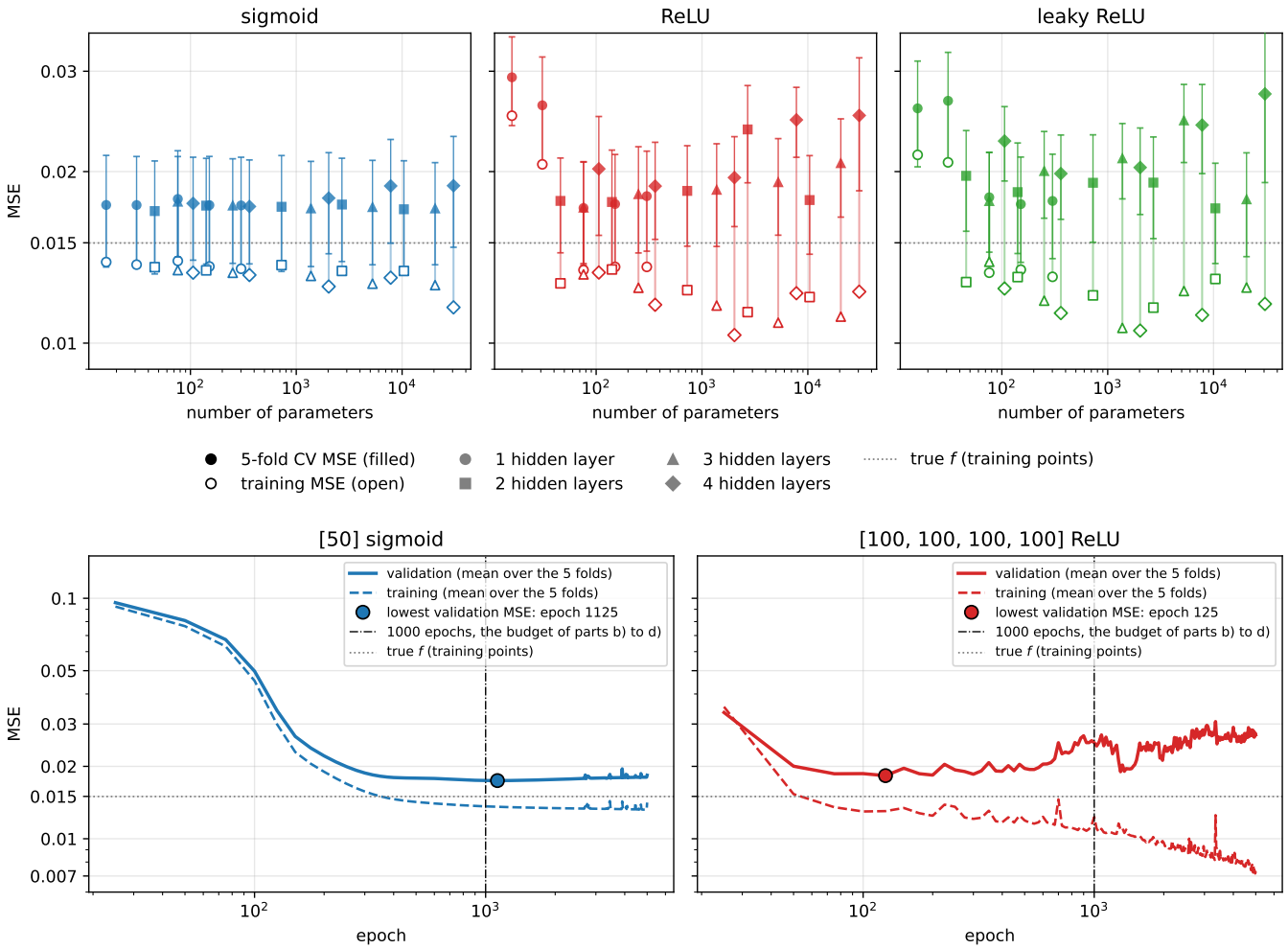


FIG. 5. Four hidden layers of 100 ReLU nodes overfit: their CV MSE is 0.0251 against a training MSE of 0.0123 (top), and their validation MSE is lowest at epoch 125, 0.0183, and reaches 0.0251 at epoch 1000 while the training MSE keeps falling, whereas the validation MSE of [50] sigmoid is lowest at epoch 1125, 0.0175 (bottom). Top: error bars of one standard error over the five folds, and faint lines joining the training and CV MSE of each network; seed 2026.

ble III). Even with matched conventions (Table III), the penalties of networks and polynomials act on different parameters and are not comparable. The networks with L_2 and L_1 score 0.0173 ± 0.0038 and 0.0175 ± 0.0040 in CV against 0.0273 ± 0.0059 for Ridge and 0.0264 ± 0.0062 for the Lasso, better by more than the standard error of each model; these CV values, however, are minima over our scans and therefore optimistic, like that of Ridge below. On the test set, used once, the networks are better by less than one standard error, and with respect to f they are much closer (0.00168 and 0.00151 against 0.00480 and 0.00502). The penalised polynomials also miss the top of the peak (about 0.83, read from Fig. 8): a penalty only shrinks [2, Eqs. (3.46)–(3.48)] or removes [2, Sec. 3.9] coefficients of a polynomial of degree 8. Selected by CV on our data, Ridge moves to degree 11 ($\lambda_b = 3 \cdot 10^{-5}$), with the lowest CV MSE of the polynomials (0.0217 ± 0.0039) but a worse test MSE (0.0219) and the worst MSE to f (0.00710): the penalty makes degree 11 usable (0.1835 in CV for OLS) without giving the shape of the peak, and the lowest CV MSE among 23×15 combinations of λ and degree is optimistic [2, Sec. 3.14].

The weights. No weight becomes exactly zero with either penalty (Appendix A): L_1 puts 6441 of the 7600 weights of [50, 50, 50, 50] below 10^{-3}, but none below 10^{-8}, and even without hidden layers our L_1 network ends $2.1 \cdot 10^{-4}$ from the Lasso solution of Scikit-Learn (notebook, Sec. b.4), while OLS and Ridge agree to rounding (Table II). Exact zeros belong to the exact minimum, which the coordinate descent of Scikit-Learn reaches by soft thresholding [2, Eqs. (3.63)–(3.64), (3.68)–(3.69)], [16], while our gradient steps hover around zero and never land on it [2, Sec. 4.14.7]: the sparsity of our L_1 network is made of very small weights, not of exact zeros.

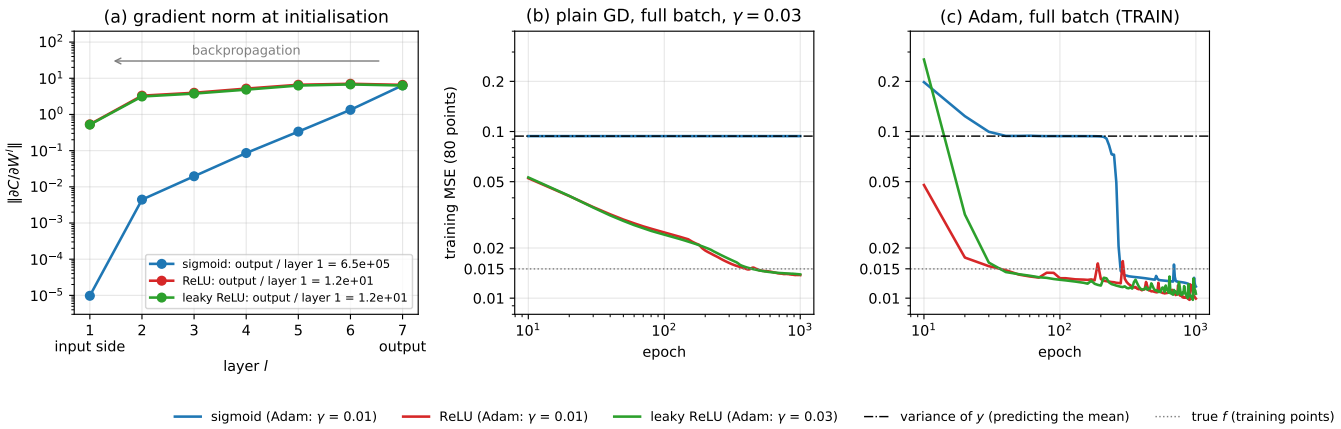


FIG. 6. With six hidden layers of 50 nodes, the norm of the weight gradient of the first layer at initialisation is smaller than that of the output layer by a factor $6.5 \cdot 10^5$ for the sigmoid and 12 for the ReLU family (a); plain GD at $\gamma = 0.03$ leaves the sigmoid network at a training MSE of 0.0939, about the variance of y, 0.0938, while the ReLU networks train (b), and Adam brings the sigmoid network to 0.0117 after 1000 epochs (c). One initialisation; seed 2026.

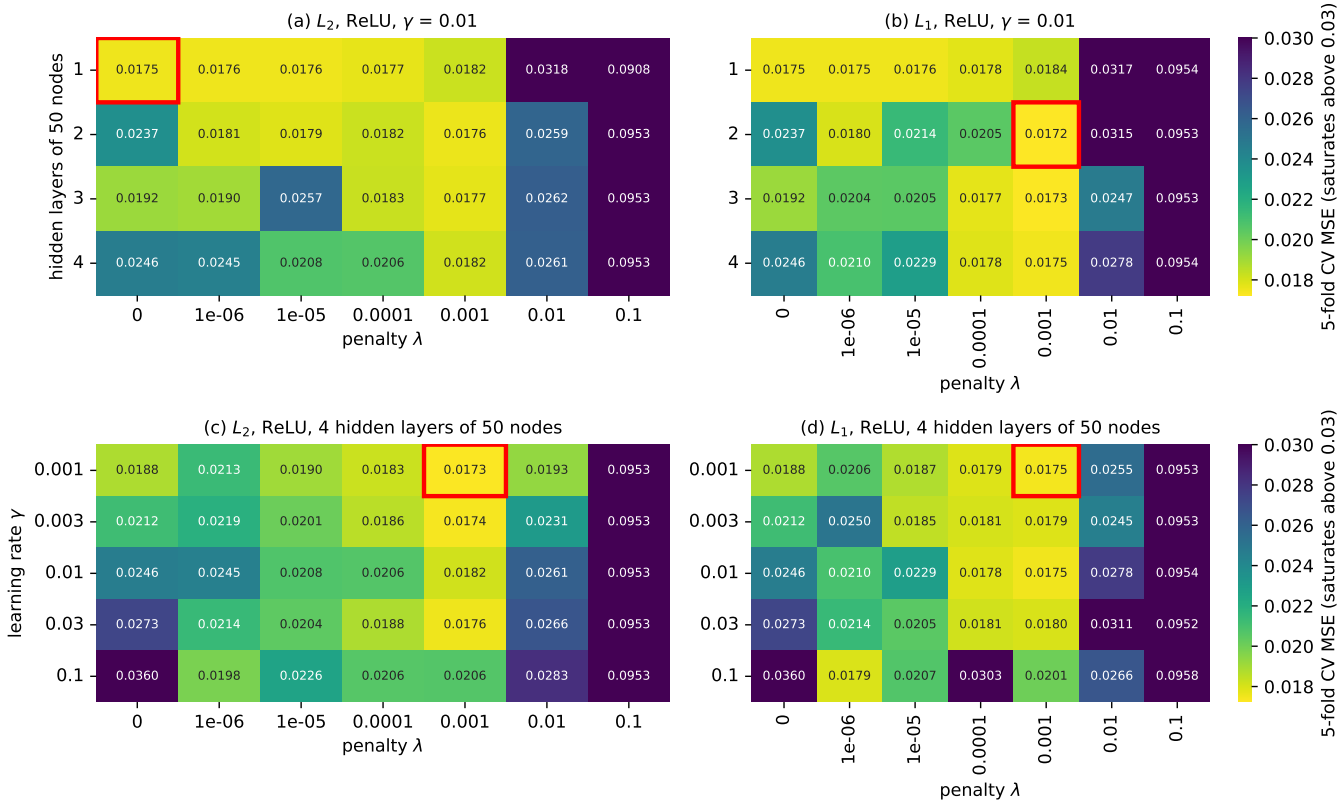


FIG. 7. With $\lambda = 10^{-3}$ the CV MSE of 2 to 4 hidden layers of 50 ReLU nodes lies between 0.0172 and 0.0182 with either penalty, against 0.0192 to 0.0246 without penalty (a, b), and for four layers the column $\lambda = 10^{-3}$ holds the lowest CV MSE at four of the five learning rates (c, d). Red frames: lowest value of each map; colours saturate above 0.03; seed 2026.

V. RESULTS: CLASSIFICATION OF MNIST

Baseline and logistic regression. Always predicting the most frequent digit gives 11.24 % on the validation set and 0.1135 on the test set. Logistic regression, our network without hidden layers ([784, 10], 7850 parameters), reaches a validation accuracy of 0.9103 with Adam at $\gamma = 10^{-2}$ and $\lambda = 0$ (Fig. 15 in Appendix B), against 0.8283 at $\gamma = 10^{-4}$ and 0.8844 at $\gamma = 0.1$; at $\gamma = 10^{-2}$ every L_2 penalty lowers it, down to 0.9028 at $\lambda = 10^{-3}$.

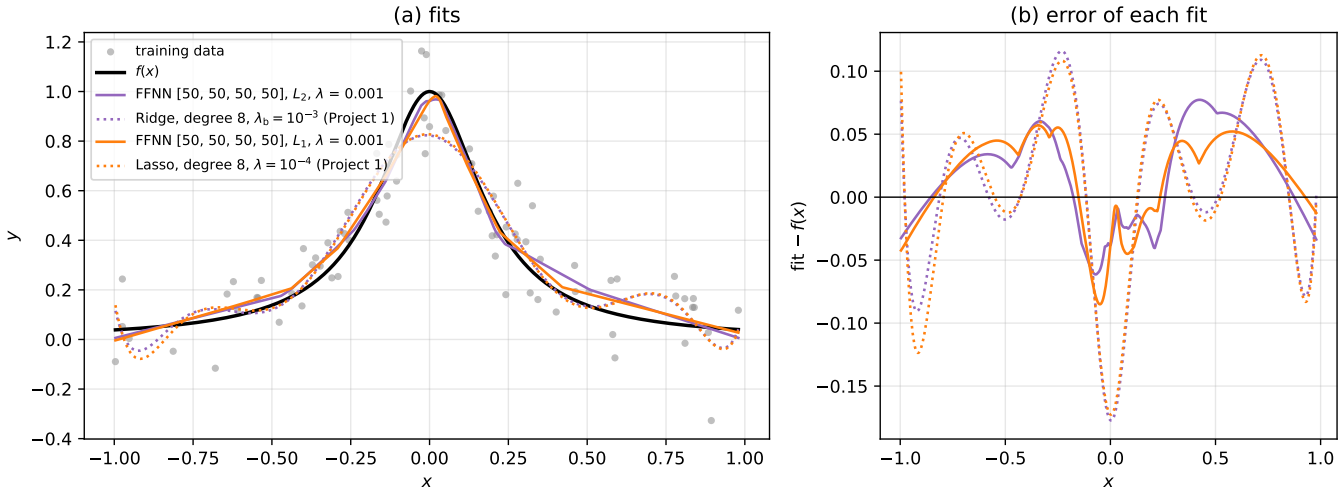


FIG. 8. The penalised networks follow the peak that Ridge and Lasso of Project 1 miss (a, b) and are closer to f, with an MSE to f of 0.00168 for L_2 and 0.00151 for L_1 against 0.00480 for Ridge and 0.00502 for the Lasso.

TABLE III. Every network scores below every polynomial model in CV MSE, 0.0168 to 0.0181 against 0.0217 to 0.0273, in test MSE, 0.0116 to 0.0168 against 0.0178 to 0.0219, and in MSE to f, 0.00081 to 0.00168 against 0.00480 to 0.00710 (notebook, Sec. f). Every row uses the same 80 training points, five folds and 20 test points (seed 2026); all our hyperparameters were chosen by CV on the training points and each test MSE was computed once, while the rows marked Project 1 keep the degree and penalty selected in Ref. [1], where OLS and Ridge were chosen on these same test points; our CV confirms OLS and the Lasso. Part: part of the project text; CV and test MSE with one standard error; λ_b: the Ridge penalty of Project 1, without the factor $1/n$; λ of the Lasso: $\alpha = \lambda/2$ in Scikit-Learn, whose cost carries a factor $1/(2n)$ [2, Sec. 3.14], [16]; rows sorted by CV MSE; bold: lowest value of each error column. The true f scores 0.0150 on the training points and 0.0096 on the test points.

model	part	CV MSE	test MSE	MSE to f	parameters
FFNN [100, 100], sigmoid	b	$\mathbf{0.0168} \pm 0.0036$	$\mathbf{0.0116} \pm 0.0043$	0.00104	10401
FFNN [5, 5], sigmoid	c	0.0171 ± 0.0038	0.0133 ± 0.0047	0.00100	46
FFNN [100, 100], leaky ReLU	c	0.0172 ± 0.0034	0.0118 ± 0.0039	0.00158	10401
FFNN [25], ReLU	c	0.0173 ± 0.0036	0.0168 ± 0.0056	0.00147	76
FFNN [50, 50], sigmoid	b	0.0173 ± 0.0038	0.0136 ± 0.0049	0.00084	2701
FFNN [50, 50, 50, 50], ReLU, L_2, $\lambda = 10^{-3}$	d	0.0173 ± 0.0038	0.0145 ± 0.0049	0.00168	7801
FFNN [100], sigmoid	b	0.0174 ± 0.0038	0.0135 ± 0.0048	0.00096	301
FFNN [50], sigmoid	b	0.0175 ± 0.0038	0.0133 ± 0.0048	0.00106	151
FFNN [50, 50, 50, 50], ReLU, L_1, $\lambda = 10^{-3}$	d	0.0175 ± 0.0040	0.0158 ± 0.0055	0.00151	7801
Scikit-Learn `MLPRegressor` [50], sigmoid	b	0.0181 ± 0.0039	0.0128 ± 0.0046	**0.00081**	151
Ridge, degree 11, $\lambda_b = 3 \cdot 10^{-5}$ (CV on our data)	d	0.0217 ± 0.0039	0.0219 ± 0.0050	0.00710	12
Lasso, degree 8, $\lambda = 10^{-4}$ (Project 1)	d	0.0264 ± 0.0062	0.0178 ± 0.0054	0.00502	9
OLS, degree 8 (Project 1)	b	0.0266 ± 0.0071	0.0187 ± 0.0048	0.00572	9
Ridge, degree 8, $\lambda_b = 10^{-3}$ (Project 1)	d	0.0273 ± 0.0059	0.0178 ± 0.0055	0.00480	9

Update rule and learning rate. For [784, 128, 10] with the ReLU, minibatches of 128 and 10 epochs on 10000 images (Fig. 9), plain SGD reaches 0.9521 at $\gamma = 1$, the edge of the grid, still rising (0.8730, 0.9309 and 0.9521 at $\gamma = 10^{-2}$, 0.1 and 1, read from the figure); RMSprop reaches 0.9529 at $\gamma = 10^{-3}$ and Adam 0.9555 at $\gamma = 10^{-2}$. At $\gamma = 1$ the adaptive rules collapse (RMSprop 0.2058, Adam 0.1042, read from the figure), since their steps are of order γ (Sec. IV A). Adam and RMSprop are close; we kept Adam for its wider range of learning rates in Fig. 2 and because RMSprop does not settle, and did not try RMSprop beyond this scan.

Penalty and architecture. The learning rate matters more than the penalty (Fig. 10, top): without penalty the accuracy is 0.9466 at $\gamma = 10^{-3}$ and 0.9555 at 10^{-2}, and at $\gamma = 0.1$ it lies between 0.8425 and 0.8913 whatever the penalty. At the two smaller learning rates, penalties up to $\lambda = 10^{-4}$ change the accuracy only in the fourth decimal (0.9463 to 0.9467 and 0.9536 to 0.9542), and at the selected $\gamma = 10^{-2}$ every penalty lowers it, so we kept

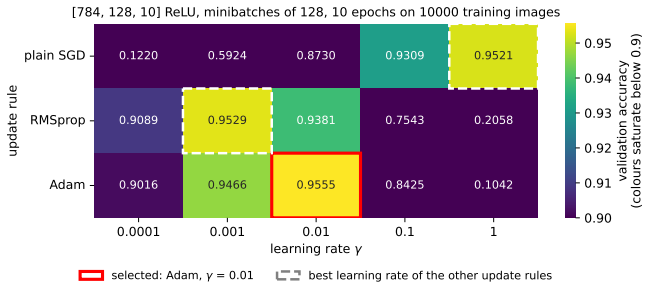


FIG. 9. Adam reaches the highest validation accuracy, 0.9555 at $\gamma = 10^{-2}$, against 0.9529 for RMSprop at $\gamma = 10^{-3}$ and 0.9521 for plain SGD at $\gamma = 1$, the edge of the grid. Colours saturate below 0.9.

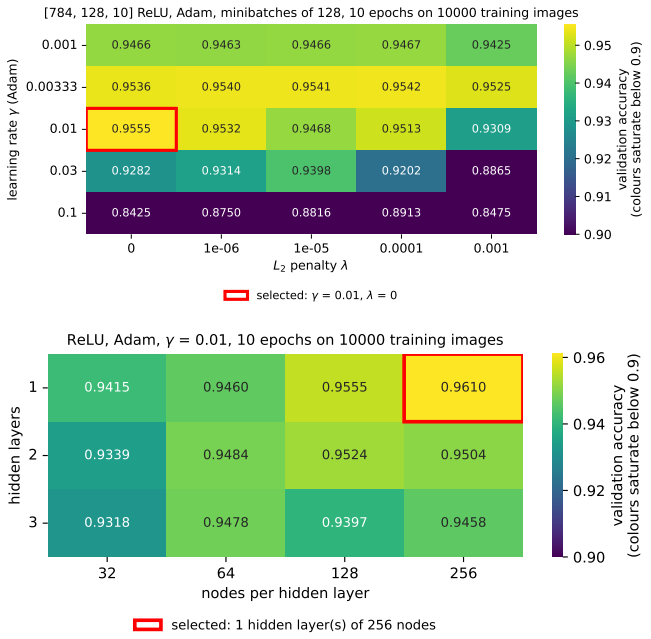


FIG. 10. Neither a penalty nor a deeper network beats the selected cells: 0.9555 without penalty at $\gamma = 10^{-2}$, against at most 0.9542 with a penalty (top), and 0.9610 for one hidden layer of 256 nodes, against at most 0.9524 with two or three layers (bottom). Colours saturate below 0.9.

$\lambda = 0$. With one hidden layer the accuracy grows with the width (Fig. 10, bottom), 0.9415, 0.9460, 0.9555 and 0.9610 from 32 to 256 nodes, 256 being the edge of the grid, and the best deeper networks, 0.9524 (two layers of 128) and 0.9478 (three of 64), stay below. We selected [256], with 203530 parameters; the choice holds at the γ selected for [128] and after 10 epochs on 10000 images, and no hyperparameter, λ included, was scanned again for the final training on 50000 images.

Activation. With [128] the best accuracies of the three activations lie between 0.9542 and 0.9555, while with [256] the ReLU (0.9634) and the leaky ReLU (0.9624) beat the sigmoid (0.9568) (Fig. 11). We selected the ReLU with $\gamma = 0.00333$, the smallest learning rate tried. With one hidden layer the error crosses a single factor f' on its way back, Eq. (7), so the vanishing gradient of Sec. IV C, which grows with depth, does not explain the lower accuracy of the sigmoid with [256].

Test set. On the test set (Table IV) our network [256] ReLU reaches 0.9806 ± 0.0014 against 0.9241 ± 0.0026 for our logistic regression, far beyond the standard errors and beyond the spread over seeds of about ± 0.2 points that the course gives for a similar network [23], which we did not measure. Our network and Scikit-Learn's `MLPClassifier` (0.9790 ± 0.0014) are close, with overlapping error bars, and so are the two logistic regressions (0.9257 ± 0.0026 for Scikit-Learn, on all 60000 training images). With 194 test errors our network lies between the dense networks [784, 100, 10] of Ref. [2, Table 10.3], with 255 to 280 errors after five epochs on all 60000 images, and the convolutional network of its listings, with 84 and 127. Early stopping kept epoch 49 of 50 for the network, so it stopped nothing, and more epochs were not tried; for logistic regression it kept epoch 15, while the validation accuracy jumps between about 0.907 and 0.922 (read from Fig. 16 in Appendix B), consistent with the cloud around the minimum of a minibatch iterate at constant γ [2, Sec. 4.7.8, Eqs. (4.50), (4.51)].

Confusion matrices. Logistic regression makes 759 errors on the 10000 test images, with a macro F_1 of 0.9231 (Fig. 12). The 5 is its hardest digit (recall 0.865) and the 8 its least reliable answer (lowest precision, 0.862); the largest confusions are a 5 read as a 3 (41 images), then 2 as 8, 5 as 8, 7 as 2 and 9 as 7 (30 each). The network makes 194 errors, with a macro F_1 of 0.9805 and precision and recall between 0.96 and 0.993 for every digit; the confusion of 5 with 3 falls from 41 to 15, that of 7 with 2 from 30 to 7, and that of 2 with 8 from 30 to 6 (read from the figure). The 5 is still its hardest digit (recall 0.965), the one we would flag to a user, while its least reliable answer is now the 3 (precision 0.96); a 9 is read as a 4 nine times, a 4 as a 9 seven times. Logistic regression has a single linear score per digit [2, Eq. (5.27)], so digits of similar shape are confused; the accuracy alone would hide which errors remain [3].

Why the hidden layer helps. Logistic regression is limited by its bias [2, Sec. 2.10]: its training accuracy is only 0.9347 (0.9224 on the validation set), more data change little (a validation accuracy of 0.9103 after 10 epochs on 10000 training images and 0.9224 at the best of 50 epochs on 50000, and a test accuracy of 0.9257 for Scikit-Learn with all 60000), and a penalty does not raise its accuracy beyond the fourth decimal. The network is logistic regression on features learned by its hidden layer [2, Sec. 8.11]; its gap between training (0.9990) and validation accuracy (0.9816) is variance [2, Sec. 8.14, Fig. 8.15], but its validation accuracy shows no downward trend over the 50 epochs (Fig. 16). Lowering the bias was worth far more than the variance it costs.

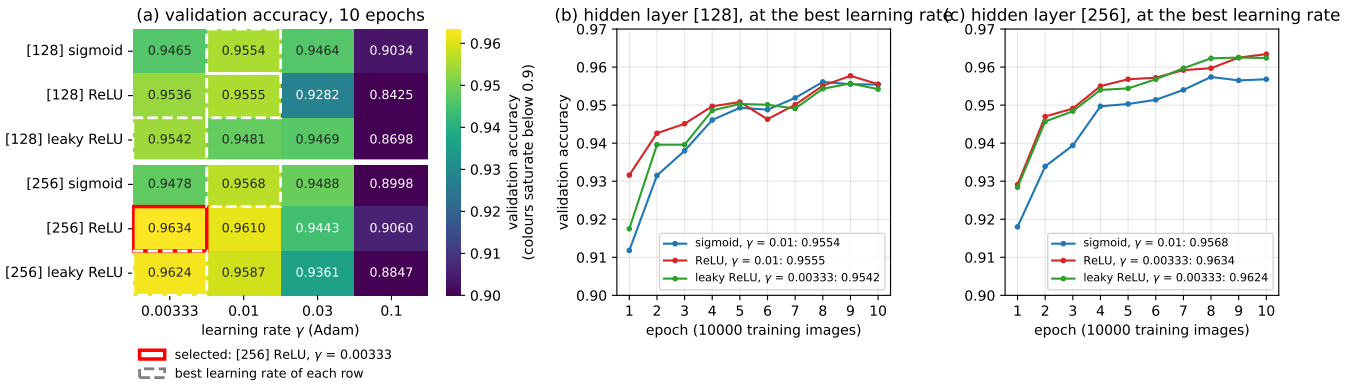


FIG. 11. For [256] the best validation accuracy of the ReLU (0.9634) and of the leaky ReLU (0.9624) is higher than that of the sigmoid (0.9568), while for [128] the best accuracies of the three activations lie between 0.9542 and 0.9555. Colours saturate below 0.9.

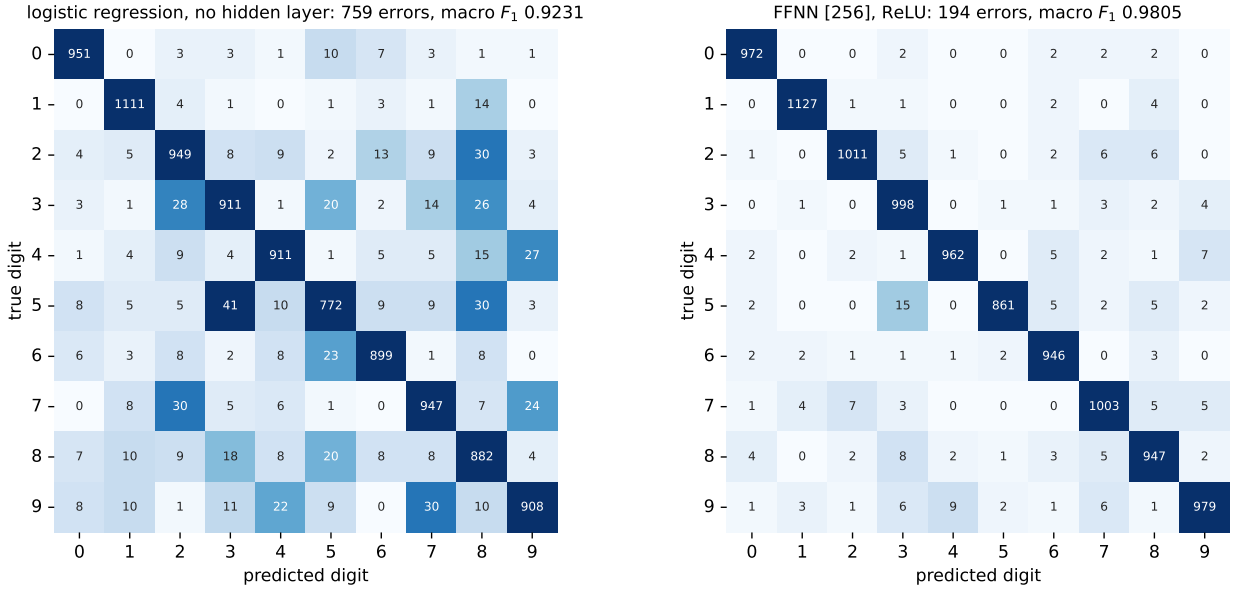


FIG. 12. The network [256] ReLU makes 194 errors on the 10000 test images (macro F_1 0.9805) against 759 for logistic regression (macro F_1 0.9231), and the largest confusion of logistic regression, a 5 read as a 3 in 41 images, falls to 15 with the network. Rows: true digit; columns: predicted digit; counts on the 10000 test images, with one colour scale for both matrices, set by the largest error, 41, so that the diagonal saturates.

VI. CRITICAL EVALUATION AND CONCLUSIONS

A. Critical evaluation

On the Runge function every network scores below every polynomial model in CV MSE and in MSE to f, while on the test set the error bars overlap (Table III); paired with OLS, the network [50] differs by -0.0054 ± 0.0028 on the test set. We call best the lowest CV MSE. Among the networks this minimum is flat: every network of Table III and every sigmoid network of Fig. 4 lies within one standard error, 0.0036, of the lowest value, 0.0168 for [100, 100] sigmoid. We therefore report this region and its simplest network [23], one hidden layer of 5 sigmoid nodes (0.0175), the choice of the one-standard-error rule [2, Eq. (3.96)], which we did not evaluate on the test set; the smallest we did evaluate is [5, 5] sigmoid (46 parameters, test MSE 0.0133 ± 0.0047). On MNIST the network is clearly the most accurate model, close to `MLPClassifier` (Table IV).

Table V compares the methods. Features chosen by hand are worth a great deal: on the Ising example of Ref. [2, Secs. 8.15, 8.16] the Lasso, "given features a physicist chose", reaches $R^2 = 0.99996$, while a network that can represent the answer exactly does not find it from four hundred configurations. A network pays off when the features are unknown, as for the pixels of MNIST [3];

TABLE IV. Our network [256] ReLU reaches a test accuracy of 0.9806 ± 0.0014, against 0.9790 ± 0.0014 for `MLPClassifier` [256] ReLU and 0.9241 ± 0.0026 for our logistic regression (notebook, Secs. e.7 and f). Our hyperparameters were chosen on the validation set, and `MLPClassifier` has those of our network: Adam, $\gamma = 0.00333$, $M = 128$, no penalty. All models were trained on the same 50000 training images, except `LogisticRegression` on all 60000, and evaluated once on the same 10000 test images. Test accuracy with one standard error; time of the final run on our machine, for our two models that of all 50 epochs. Epochs: for our models the epoch kept by early stopping; for `MLPClassifier` the epochs it was trained for, set to our kept epoch; for `LogisticRegression` the iterations of its saga solver. Bold: highest test accuracy.

model	test accuracy	validation	training	parameters	epochs	time (s)
FFNN [256], ReLU (ours)	**0.9806** $\pm$ 0.0014	0.9816	0.9990	203530	49	366
Scikit-Learn `MLPClassifier` [256], ReLU	0.9790 ± 0.0014	0.9795	0.9982	203530	49	299
Scikit-Learn `LogisticRegression`, saga	0.9257 ± 0.0026	n/a	n/a	7850	511	n/a
logistic regression (ours)	0.9241 ± 0.0026	0.9224	0.9347	7850	15	8
most frequent digit	0.1135	n/a	n/a	0	n/a	0

TABLE V. The network is the most accurate model on both problems, narrowly on the Runge function, the only one that is not convex and the one with the most hyperparameters, and on MNIST it takes 366 s against 8 s for logistic regression: strengths and weaknesses of the methods, from our results and from Ref. [2, Secs. 4.7.2, 5.4, 8.15]; times of the final run on our machine.

	neural network	OLS, Ridge, Lasso	logistic regression
accuracy	best on both problems, narrowly on the Runge function (Tables III, IV)	limited by the polynomial basis on the Runge function	limited by its bias on MNIST
cost	iterative: 0.68 s per 5-fold CV of [50] with Adam and the full batch, 366 s on MNIST	closed form for OLS and Ridge, "the cheapest option of all" [2, Sec. 4.7.2]; coordinate descent for the Lasso	iterative: 8 s on MNIST
hyperparameters	γ, update rule, epochs, architecture, activation, λ	degree, and λ for Ridge and Lasso	γ, λ, epochs
convexity	not convex: "results depend on initialisation and on the random seed" [2, Sec. 8.15], and the irregular cell of Fig. 4 is compatible with it	convex, deterministic	convex [2, Sec. 5.4]
interpretability	"largely uninterpretable" [2, Sec. 8.15]: 151 parameters for [50], without standard errors	variances of the coefficients for OLS and Ridge [2, Eqs. (3.32), (3.34), (3.51)], bootstrap for the Lasso [2, Sec. 3.14]	one weight vector per digit
extrapolation	networks "extrapolate unpredictably outside the range of the training data" [2, Sec. 8.15]; not tested	a polynomial grows without bound	not tested
data	networks "need large amounts of labelled data" [2, Sec. 8.15]: a clear gain on 50000 images, a narrow one on 80 points	suited to few data with the right basis	n/a

on a smooth function of one variable its gain was narrow.

The learning rate mattered more than the update rule (Figs. 2 and 9). Plain GD with the full batch stayed outside one standard error of the best (0.0301 at $\gamma = 0.1$) and left the deep sigmoid network at the mean; RMSprop was erratic at large γ; Adam had the widest range, from $\gamma = 3.2 \cdot 10^{-3}$ to 0.32 for [50]; minibatches converged in fewer epochs but took longer per epoch. For a new problem we would use Adam and scan γ from 10^{-3} to $3 \cdot 10^{-2}$, the range of its best values in all our scans.

B. Conclusions

The network beat the polynomials on the Runge function, narrowly, because it follows the narrow peak that a polynomial basis of degree 8 misses, with or without a penalty; on MNIST a hidden layer lowered the bias of logistic regression and raised the test accuracy from 0.9241 to 0.9806. Matching the output activation to the cost kept the output error large where the network was confidently wrong, and the factor $\sigma' \leq 1/4$ that spoils the mismatched pair returned in deep sigmoid networks as a vanishing gradient, which the ReLU family avoids and which Adam overcame in our test, unlike plain gradient

descent. Deep and wide ReLU networks overfit, and an L_1 or L_2 penalty brought those with 50 nodes per layer back to the level of the best small networks. Our gradients agreed with automatic differentiation to rounding, and the network without hidden layers reproduced the closed forms of Project 1.

We could not settle the ranking among the regression networks, nor between Adam and RMSprop on MNIST: every network result comes from one seed, and the spread between seeds can exceed the differences between methods [2, Table 8.2]. The regression rests on 100 points split 80/20 and on a fixed budget of 1000 epochs, the MNIST scans on 10000 images and 10 epochs. Several choices lie at the edge of their grid ($\gamma = 10^{-3}$ in Fig. 7, plain SGD at $\gamma = 1$, the width 256 and $\gamma = 0.00333$), and the kept epoch is 49 of 50. The penalties were tried only with the ReLU and 50 nodes per layer, extrapolation outside $[-1, 1]$ was not tested, and we did not carry out the optional bias-variance analysis.

Four extensions follow from these limits. Several seeds and repeated cross-validation would measure the spread between seeds, which in Ref. [2, Table 8.2] is larger than the spread among the non-saturating activations. A learning rate schedule [2, Sec. 4.7.8] and early stopping in the regression study, with wider grids, would replace the fixed budget of epochs and show where the optima now at the edges lie. Soft thresholding [2, Eq. (3.63)] in the update of the L_1 network would give exact zeros. For MNIST, a convolutional network is the step towards the accuracy above 99 % quoted in the project text [3].

Appendix A: Additional regression results

Figure 13 shows the fits of the best network of each activation of Fig. 4, [5, 5] sigmoid, [25] ReLU and [100, 100] leaky ReLU, written 2×5, 1×25 and 2×100 in the legend with their test MSE. The test MSE on 20 points does not rank them: [100, 100] leaky ReLU has the lowest test MSE (0.0118 ± 0.0039) but the highest MSE to f of the three (0.00158), against 0.0133 ± 0.0047 and 0.00100 for [5, 5] sigmoid and 0.0168 ± 0.0056 and 0.00147 for [25] ReLU, and the ReLU fits are visibly piecewise linear.

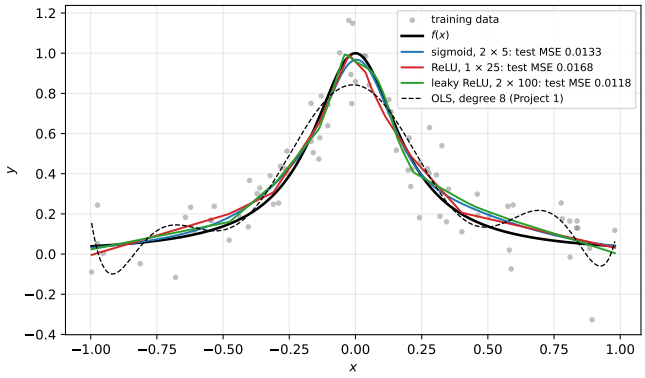


FIG. 13. Among the best networks of the three activations, [100, 100] leaky ReLU has the lowest test MSE, 0.0118, but the highest MSE to f, 0.00158, against 0.0133 and 0.00100 for [5, 5] sigmoid and 0.0168 and 0.00147 for [25] ReLU.

Figure 14 shows the sizes of the weights discussed in Sec. IV D, for the two networks selected in Sec. IV D and for the same network trained without penalty at the same learning rate (notebook, Sec. d.4). Of the 7600 weights, 28 are below 10^{-3} without penalty. With L_2, 2624 are below 10^{-3} and 1275 below 10^{-8}, down to $4.0 \cdot 10^{-27}$. With L_1, 6441 are below 10^{-3}, most of them above 10^{-6}, and none below 10^{-8}, the smallest being $2.5 \cdot 10^{-8}$. Our tests do not explain why L_2 drives more weights than L_1 below 10^{-8}. The sum of the squared weights is 439.853 without penalty, 110.639 with L_2 and 102.308 with L_1; the sum of their absolute values is 1316.924, 366.106 and 183.618.

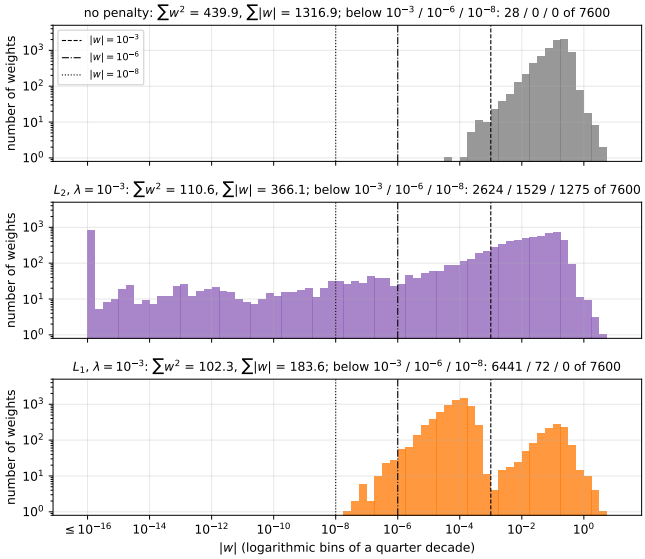


FIG. 14. No weight is exactly zero with either penalty: the smallest is $4.0 \cdot 10^{-27}$ with L_2 and $2.5 \cdot 10^{-8}$ with L_1, although L_1 puts 6441 of the 7600 weights below 10^{-3}. Logarithmic bins of a quarter decade; weights below 10^{-16} are counted in the leftmost bin.

Appendix B: Additional classification results

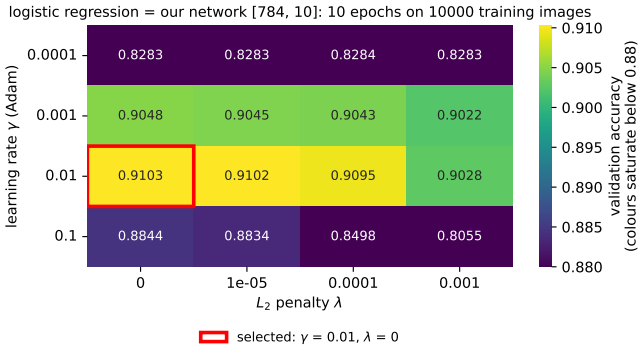


FIG. 15. Logistic regression reaches its highest validation accuracy, 0.9103, at $\gamma = 10^{-2}$ without penalty, and every penalty lowers it at that learning rate. Colours saturate below 0.88.

Figures 15 and 16, discussed in Sec. V, show the scan of logistic regression and the validation accuracy of the two final models of Table IV after every epoch: the curve of logistic regression jumps from epoch to epoch at constant γ, while that of the network oscillates with no downward trend.

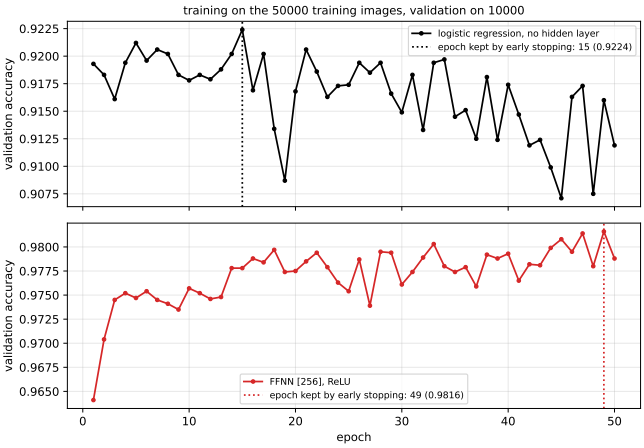


FIG. 16. Early stopping keeps epoch 15 for logistic regression, with a validation accuracy of 0.9224, and epoch 49 of 50 for the network [256] ReLU, with 0.9816.

Appendix C: Use of AI/LLM Tools

AI was used to draft the code and to organise the available code from the lecture material. Every line of code was commented, explained, checked multiple times and, where necessary, corrected and modified by hand, against the task, the lecture material and Project 1. We used the drafted and corrected code to go back to the lecture material that explains the theory behind it, so that we could see the concepts of the course put into practice. AI was also used to translate our own reasoning, reflections and doubts into academic English, to put our ideas, calculations, implementations and results into LaTeX as a well-organised report, and to organise our considerations on the practical applications of the theory of the course, and our reflections, into a scientific report.

Tools used. Claude (Claude Opus 5.5, Anthropic), accessed through claude.ai in October and November 2026.

Text writing and editing. Table VI gives one level for every section of the report: 0, no help; 1, language only; 2, editorial (the model rewrote our sentences, the ideas are ours); 3, generative (the model drafted substantial text from our prompts). For every section at level 3 the prompt was the same: we gave Claude our working document, with the rules of the course and of our group and the agreed content of every section, the executed notebook and Ref. [2], and asked it to write the report in LaTeX following that document, to check every number against the output of the notebook and every quotation against Ref. [2], and to have layout, compliance and citations checked by independent sessions of the same model. The working document and the text cells of the notebook were themselves drafted with Claude from our decisions and from our sessions on the notebook, and checked by us against the outputs, the sources and the project text. The last column of Table VI gives the material from which each section was drafted.

TABLE VI. Level of LLM contribution to the text of each section (0 none, 1 language only, 2 editorial, 3 generative).

Section	Level	Notes
Abstract	3	drafted from the results of the notebook and our considerations
Introduction	3	drafted from the project text, the lecture notes and Project 1
Theory and methods	3	drafted from the text cells of the notebook and the lecture notes
Implementation and verification	3	drafted from the course and project1 code and their printed checks
Results: regression	3	drafted from the outputs and our reasoning in the text cells of parts b) to d) of the notebook
Results: classification	3	drafted from the outputs and our reasoning in the text cells of part e) of the notebook
Critical evaluation and conclusions	3	drafted from the results, our reasoning, notes on the notebook and the caveats noted in part f) of the notebook
Appendices A and B	3	drafted from the outputs of the notebook
This appendix	3	first paragraph written by us; the rest drafted from the LLM notes in the notebook
Captions, tables and Algorithm 1	3	drafted from the outputs and the code of the notebook
References	3	drafted from the reference cell of the notebook and checked against each source

Code generation and assistance. Table VII gives one level for every code file: 0, none; 1, debugging; 2, snippets; 3, skeleton; 4, substantial. Every function written with Claude carries an `LLM-assisted` tag in its docstring and every code cell of ours an LLM note above it; these tags, the appendix of the notebook and this appendix tell the same story. Claude also reviewed the notebook against the course repository, Project 1 and the outputs, and proposed corrections that we evaluated and applied. Every number of this report was compared with the output of the final run, and every reference with its source.

TABLE VII. Level of LLM contribution to the code (0 none, 1 debugging, 2 snippet, 3 skeleton, 4 substantial).

File	Level	Description
`Project2.ipynb`, parts a) to f)	4	Claude wrote the code cells of all parts, starting from the course code of weeks 41 and 42, Project 1 and the starter cells of the project text, as stated above every cell. The starter cells are course code (level 0), and the functions reused from Project 1 are declared where they are used.
`Project2.py`	4	Script exported from the notebook: the same code.

REFERENCES

[1] P. Monacelli, G. Kotrotsios, G. Spreafico, and E. Valenti, Project 1, FYS-STK3155/4155 report and code, University of Oslo, https://github.com/georgkot/FYS-STK4155 (2026).

[2] M. Hjorth-Jensen, *Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning* (Lecture notes for FYS-STK3155/4155, University of Oslo, 2026) version of October 25, 2026, `doc/BookML/Lecturebook.pdf` in https://github.com/EducationalMaterialUiO/MachineLearningUiO.

[3] M. Hjorth-Jensen, Project 2 on Machine Learning. Classification and regression: writing our own neural network code, FYS-STK3155/4155, University of Oslo, `doc/Projects/2026/Project2/Project2.pdf` in https://github.com/EducationalMaterialUiO/MachineLearningUiO (2026), published October 6, 2026.

[4] M. Leshno, V. Y. Lin, A. Pinkus, and S. Schocken, Multilayer feedforward networks with a nonpolynomial activation function can approximate any function, Neural Networks **6**, 861 (1993).

[5] A. E. Hoerl and R. W. Kennard, Ridge regression: Biased estimation for nonorthogonal problems, Technometrics **12**, 55 (1970).

[6] R. Tibshirani, Regression shrinkage and selection via the lasso, Journal of the Royal Statistical Society: Series B (Methodological) **58**, 267 (1996).

[7] Y. LeCun, C. Cortes, and C. J. C. Burges, MNIST handwritten digit database, AT&T Labs, http://yann.lecun.com/exdb/mnist (2010).

[8] C. R. Harris *et al.*, Array programming with NumPy, Nature **585**, 357 (2020).

[9] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, Learning representations by back-propagating errors, Nature **323**, 533 (1986).

[10] J. Bradbury *et al.*, JAX: composable transformations of Python+NumPy programs, https://github.com/jax-ml/jax (2018).

[11] F. Pedregosa *et al.*, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research **12**,

2825 (2011).
[12] T. Tieleman and G. Hinton, Lecture 6.5, rmsprop: Divide the gradient by a running average of its recent magnitude, COURSERA: Neural Networks for Machine Learning (2012).
[13] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization (2014), published at the 3rd International Conference on Learning Representations (ICLR), San Diego, 2015, arXiv:1412.6980.
[14] M. Hjorth-Jensen, Week 41, Monday lecture, Jupyter notebook `doc/WeeklyMaterial/week41/week41monday.ipynb`, FYS-STK3155/4155, University of Oslo, in https://github.com/EducationalMaterialUiO/MachineLearningUiO (2026).
[15] P. Monacelli, G. Kotrotsios, G. Spreafico, and E. Valenti, FYS-STK4155, Project 2: code, executed notebook and figures, https://github.com/pietromonacelli/FYS-STK4155-Project2 (2026).
[16] M. Hjorth-Jensen, Week 40, Tuesday session, slides `doc/WeeklyMaterial/week40/week40tuesday.pdf`, FYS-STK3155/4155, University of Oslo, in https://github.com/EducationalMaterialUiO/MachineLearningUiO (2026).
[17] M. Hjorth-Jensen, Exercises week 41, Jupyter notebook `doc/WeeklyMaterial/week41/exercisesweek41.ipynb`, FYS-STK3155/4155, University of Oslo, in https://github.com/EducationalMaterialUiO/MachineLearningUiO (2026).
[18] K. He, X. Zhang, S. Ren, and J. Sun, Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification, in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (2015) pp. 1026–1034, arXiv:1502.01852.
[19] X. Glorot and Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, Proceedings of Machine Learning Research, Vol. 9, edited by Y. W. Teh and M. Titterington (PMLR, Chia Laguna Resort, Sardinia, Italy, 2010) pp. 249–256.
[20] M. Hjorth-Jensen, Exercises week 42, solutions, Jupyter notebook `doc/WeeklyMaterial/week42/exercisesweek42_solutions.ipynb`, FYS-STK3155/4155, University of Oslo, in https://github.com/EducationalMaterialUiO/MachineLearningUiO (2026).
[21] J. D. Hunter, Matplotlib: A 2D graphics environment, Computing in Science & Engineering **9**, 90 (2007).
[22] M. L. Waskom, seaborn: statistical data visualization, Journal of Open Source Software **6**, 3021 (2021).
[23] M. Hjorth-Jensen, Week 45, Tuesday session, slides `doc/WeeklyMaterial/week45/week45tuesday.pdf`, FYS-STK3155/4155, University of Oslo, in https://github.com/EducationalMaterialUiO/MachineLearningUiO (2026).